

1.队列

queue ()

队列 (queue)，是一种先进先出的数据结构

```
1  std::queue<T> q;  
2  q.push(x); // 元素 x 入队  
3  q.pop(); // 元素 x 出队  
4  q.top(); // 返回栈顶元素  
5  q.front(); // 返回队列的队首元素  
6  q.back(); // 返回队列的队尾元素  
7  q.empty(); // 检查是否为空,空返回1否则返回0  
8  q.size(); // 获取大小
```

deque ()

双端队列 (deque)，可以理解为一个双端数组

```
1  std::deque<int> dq;  
2  dq.push_back(x); // 在末尾加入一个元素  
3  dq.pop_back(); // 删除末尾的元素  
4  dq.push_front(x); // 在开头加入一个元素  
5  dq.pop_front(); // 删除开头的元素  
6  dq.front(); // 返回数组的开头元素  
7  dq.back(); // 返回数组的末尾元素  
8  // 中括号随机访问, i 是下标  
9  dq[i] = x;  
10 dq.empty(); // 检查是否为空,空返回1否则返回0  
11 dq.size(); // 获取大小
```

priority_queue (<priority_queue>)

优先队列 (priority_queue)，也是一种堆，可以方便获取最值。

```
1  std::priority_queue<int> pq; // C++默认大根堆 栈顶元素永远最大 d  
2  std::priority_queue<int, std::vector<int>, std::greater<int>> pq; // 小根堆  
   栈顶元素永远最小 单调递减  
3  pq.push(x); // 将元素 x 放入堆中  
4  pq.pop(); // 将堆顶元素删除  
5  pq.top(); // 返回堆顶元素, 大根堆返回最大值, 小根堆返回最小  
6  pq.empty(); // 检查是否为空,空返回1否则返回0  
7  pq.size(); // 获取大小
```

2.容器

2.1vector ()

- 定义

```
1 std::vector<T> vec; // 不写任何参数, 创建一个长度为 0, 没有元素的数组
2 std::vector<T> vec(n, val); // 创建一个长度为 n, 全部元素均为 val 的数组, val可以不
    写, 不写时填 T 的默认构造
3 std::vector<T> vec(vec_); // 拷贝构造函数, 把 vec_ 的所有内容拷贝过
4
5 //二维vector的定义, n*m大小
6 std::vector<std::vector<T>> vec(n, std::vector<T>(m));
7 //遍历用for即可
```

- 常用成员

```
1 std::vector<T> vec; // 先提前声明一个 T 类型的 vector
2 vec.push_back(x); // 最后放入一个元素 x
3 vec.pop_back(); // 去除最后一个元素
4 // 中括号随机访问, 这里 i 是一个下标
5 vec[i] = x;
6 vec.back(); // 返回 vector 最后一个元素
7 vec.clear(); //清空vector
8 vec.size(); //获取大小
```

注意: 用push_back会引发动态扩容

2.2set ()

集合 (set), 满足所有元素在里面只会出现至多一次且默认有序

-

```
1 std::set<T> s; // 构造一个空集合
2 std::set<T> s(s_); // 将集合 s_ 的内容复制到 s 中
```

-

```
1 std::set<T> s;
2 s.insert(x); // 将 x 放入集合中, 如果已经有了, 不进行任何操作
3 s.count(x); // 返回 x 在集合中的出现次数, 由于集合的特性, 可以理解为判断 x 是否存在集合
    中
4 s.erase(x); // 删除 x
5 s.find(x); // 返回一个指向元素 x 的迭代器, 找不到则返回 s.end()
6 s.empty(); //检查是否为空, 空返回1否则返回0
7 s.size(); //获取大小
```

2.3multiset ()

多重集 (multiset), 也是一个集合, 但是一种元素可以出现多次

-

```
1 std::multiset<T> s;  
2 s.count(x); // 返回 x 在 s 中的出现次数  
3 s.erase(x); // 删除 x 在 s 中的 所有 出现  
4 s.extract(x); // 删除 x 在 s 中的一个出现  
5 s.find(x); // 返回 x 在 s 中的一个出现的迭代器  
6 s.empty(); // 检查是否为空, 空返回1否则返回0  
7 s.size(); // 获取大小
```

2.4pair()

对组 (pair), 存放两个信息: first, second, 相当于一个存放两个变量的类

当一个函数需要返回2个数据的时候, 可以选择pair

-

```
1 std::pair<T1, T2> p; // T1和T2b  
2 p1.first; // 返回对象p1中名为first的公有数据成员  
3 p1.second; // 返回对象p1中名为second的公有数据成员
```

2.5map ()

`map` 容器中的元素是按照键的顺序自动排序的, 这使得它非常适合需要快速查找和有序数据的场景。

-

```
1 std::map<key_type, value_type> myMap  
2 //可声明多维map, 例  
3 std::map<int, map<int, int>> myMap;  
4 //遍历  
5 for (auto [key, value] : f) {  
6     std::cout << key << " " << value << "\n";  
7 }
```

-

```
1 mp[x]; // 返回键x对应的值  
2 myMap.erase(key); // 清楚key对应的元素  
3 myMap.clear(); // 清空容器  
4 myMap.size(); // h  
5 myMap.count("Bob"); // key 是否存在存在返回1, 否则返回0
```

2.6 stack ()

栈 (stack), 一种后进先出的数据结构, 能高效匹配括号, 也能处理递归问题等

-

```

1 std::stack<T> st;
2 st.push(x) //将元素x压入栈中
3 st.top(); //返回栈顶元素，但不对其进行操作
4 st.pop(); //移除栈顶元素
5 s.empty(); //检查是否为空，空返回1否则返回0
6 s.size(); //获取大小

```

3 模板

3.1 单调队列 滑动窗口

```

1     for(int i=1;i<=n;i++){
2         while((!dq.empty()) && a[dq.back()] <= a[i])
dq.pop_back();
3         dq.push_back(i);
4         if(dq.front() == i - k){
5             dq.pop_front(); //移除过期数据
6         }
7         if(i>=k)
8             cout << a[dq.front()] << " "; //输出窗口每次移动的最大值
9     } //输出长度为k窗口下的最大值

```

3.1.1 找出一定范围内不定长度的最大值

[P1714 切蛋糕 - 洛谷](#)

找出位于m滑块下的不定长最大

简单思考可以得到，若使维护前缀和单调递增，那么得到的答案一定最优

```

1 deque<int> dq;
2 dq.push_back(0); //初始放入索引0
3 int ans = -1e8; //如果出现全负的数据，需要（此题不需要）
4 for (int i = 1; i <= n; i++) {
5     // 维护窗口大小不超过 m，每次最多只有一个过期元素，所以可以用if，但用while更安全
6     // 这里使用if是因为我们知道每次窗口移动一步，最多只有一个过期元素
7     while (!dq.empty() && dq.front() < i - m) {
8         dq.pop_front();
9     }
10    // 更新答案
11    if (!dq.empty()) {
12        ans = max(ans, prefix[i] - prefix[dq.front()]);
13    }
14    // 维护队列单调递增，这里必须用while，因为可能弹出多个
15    while (!dq.empty() && prefix[i] <= prefix[dq.back()]) {
16        dq.pop_back();
17    } //保证前缀和总是单调递增
18    dq.push_back(i);
19 }

```

3.2 单调栈

```

1  for (int i = 1; i <= n; i++)
2      {
3          while ((!st.empty()) && a[st.top()] < a[i]) //单调递减的单调栈
4              {
5                  int index = st.top();
6                  st.pop();
7                  ans[index] = i;
8              }
9          st.push(i);
10     }

```

3.3 二分

```

1      ll r = max;
2      ll l = 1;
3      ll ans = 0;
4
5      while(l<=r){
6          ll temp = 0;
7          ll mid = (l + r) / 2;
8
9          for(int i = 0;i<n;i++){
10             temp += a[i] / mid;
11         }
12
13         if(temp < k){
14             r = mid - 1;
15         }else{
16             l = mid + 1;
17             ans = mid;
18         }
19     }
20 }

```

3.4 三分

适用场景：

用于在单峰函数（先增后减）或单谷函数（先减后增）上寻找极值。

- 如果函数是**凸的 (Convex)**（像 U 形）：求最小值。
- 如果函数是**凹的 (Concave)**（像 $\cap$ 形）：求最大值。

注意：函数必须严格单调，若存在平坦区域（平台），三分可能会失效。

3.4.1 实数域三分 (浮点数)

这是最常用的场景（如几何题、物理题）。建议使用**固定迭代次数法**，比 `while(r-l > eps)` 更快且不易死循环。

```

1      // 这里的 check 函数即题目中需要求极值的函数
2      double check(double x) {
3          double res = 0;

```

```

4      // ... 计算逻辑 ...
5      return res;
6  }
7
8  void solve() {
9      double l = 0, r = 1e9; // 根据题目范围设定
10
11     // 【求最小值】 (U形函数)
12     // 循环 100 次可以将精度控制在极高范围, 通常优于设置 eps
13     for (int i = 0; i < 100; i++) {
14         double m1 = l + (r - l) / 3;
15         double m2 = r - (r - l) / 3;
16
17         // 如果 m1 处的函数值更小, 说明极小值在 m2 左侧 (舍弃右边)
18         // 注意: 求最大值时, 符号改为 >
19         if (check(m1) < check(m2)) {
20             r = m2;
21         } else {
22             l = m1;
23         }
24     }
25
26     // 最终结果 l 和 r 几乎相等, 输出 l 或 r 均可
27     printf("%.10f\n", l);
28 }
29

```

3.4.2 整数域三分

当坐标必须是整数时, 由于整除截断问题, m_1 和 m_2 可能会重合。

通用策略: 三分将范围缩小到很小 (例如区间长度小于 3), 然后暴力枚举剩余的几个点。

code C++downloadcontent_copyexpand_less

```

1  long long solve() {
2      long long l = 0, r = 1e9; // 假设求最大值
3
4      // 1. 三分缩小范围, 直到区间长度很小 (比如 <= 2)
5      while (r - l > 2) {
6          long long m1 = l + (r - l) / 3;
7          long long m2 = r - (r - l) / 3;
8
9          // 求最大值 (凹函数 n)
10         // 谁小删谁 (m1 比较小, 说明峰值在 m1 右侧)
11         if (check(m1) < check(m2)) {
12             l = m1;
13         } else {
14             r = m2;
15         }
16     }
17
18     // 2. 此时区间 [l, r] 只剩下 l, l+1, r 等 2~3 个点
19     // 直接暴力求这几个点的最大值, 绝对不会漏, 也不会死循环
20     long long ans = -1e18; // 初始极小值
21     for (long long i = l; i <= r; i++) {

```

```

22     ans = max(ans, check(i));
23 }
24
25 return ans;
26 }

```

3.5 字符串

3.5.1 统计字符串个数

子串:对于字符串 s 与 t , 如果存在 l 与 r 满足 $1 \leq l \leq r \leq n$ 且 $t = s_l s_{l+1} \cdots s_{r-1} s_r$, 那么定义 t 为 s 的子串。例如, “garo” 是 “kangaroo” 的子串, 而 “ko” 不是 “kangaroo” 的子串。

```

1 int cout_find( string s, string p){
2     int cnt = 0;
3     size_t pos = 0;
4     while((pos = s.find(p,pos)) != string::npos){
5         cnt++;
6         pos += 1;
7     }
8     return cnt;
9 }

```

3.5.2统计子序列个数

子序列:对于字符串 s 与 t , 字符串 t 在字符串 s 中于子序列的形式出现, 意味着, 字符串 t 可由 s 删除若干个字符得到 (也可能是0个)

```

1 int Count(string s,string t){
2     vector<ll> dp(t.length()+1,);
3     dp[0] = 1;
4     for(int i = 0;i<s.length();i++){
5         for(int j = t.length();j;j--){
6             if(s[i] == t[j-1]){
7                 dp[j] += dp[j-1];
8             }
9         }
10    } //dp[j] 的含义是: 目标字符串  $t$  的“前  $j$  个字符”在当前扫描过的  $s$  中作为子序列出现了多少次。
11    return dp[t.length()];
12 }//从后往前遍历 $t$ 后,  $dp$ 里分别表示的就是对应位置的前字符串能在 $s$ 中组成几个
13 //例如  $s="babg", t="bag"$ 
14 //遍历完后,  $dp$ 为 $[1,2,1,1]$   $dp[1]=2$ , 表示" $b$ "在 $s$ 中有两个,  $dp[2]=1$ , 表示" $ba$ "在 $s$ 中有1个

```

3.5.3 字典树

利用字符串的公共前缀来减少查询时间，最大限度地减少无谓的字符串比较。

```
1  const int N = 100010; // 根据题目总字符长度设定
2  int trie[N][26];      // 假设只存小写字母，每个节点最多26个子节点
3  int cnt[N];           // 计数器
4  int idx;               // 节点分配器，从0或1开始
5
6  // 插入字符串
7  void insert(string s) {
8      int p = 0; // p代表当前所在的节点编号，0是根节点
9      for (int i = 0; i < s.size(); i++) {
10         int u = s[i] - 'a'; // 将字符转化为 0-25 的数字
11         if (!trie[p][u]) trie[p][u] = ++idx; // 如果没有路，这就新建一条路
12         p = trie[p][u]; // 走到下一个节点
13         cnt[p]++;       // 【重点】这里记录有多少个单词经过了这个节点
14     }
15 }
16
17 // 查询前缀出现次数（例如查 "ca" 是多少个单词的前缀）
18 int query(string s) {
19     int p = 0;
20     for (int i = 0; i < s.size(); i++) {
21         int u = s[i] - 'a';
22         if (!trie[p][u]) return 0; // 路断了，说明不存在这个前缀
23         p = trie[p][u];
24     }
25     return cnt[p]; // 返回经过这个节点的数量
26 }
```

3.5.4 LCS && LIS

最长公共子序列(LCS)

```
1  int dp[5010][3010];
2  string s, t;
3  cin >> s >> t;
4
5  for (int i = 1; i <= s.length(); i++)
6  {
7      for (int j = 1; j <= t.length(); j++)
8      {
9          if (s[i-1] == t[j-1])
10         {
11             dp[i][j] = dp[i-1][j-1] + 1;
12         }
13         else
14         {
15             dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
16         }
17     }
18 }
19 cout<<dp[s.length()][t.length()]; //输出最长公共子序列长度
```



```

20
21 //反向追踪出最长的公共子序列
22 string res = "";
23 int i = s.length(), j = t.length();
24 while(i>0 && j>0){
25     if(s[i - 1] == t[j-1] ){
26         res+=s[i-1];
27         i--,j--;
28     }else{
29         if(dp[i-1][j] >= dp[i][j-1]) i--;
30         else j--;
31     }
32 }
33 cout<<res;
34

```

最长上升子序列(LIS)

设计 dp_i 为以 a_i 为结尾的最长上升子序列，计算时，尝试将 a_i 接到之前的最长不上降子序列后面

```

1  dp[1] = 1;
2  ll ans = 1;
3  for(int i = 2; i <= n; i++){
4      dp[i] = 1;
5      for(int j = 1; j < i; j++){
6          if(a[j] < a[i]){
7              dp[i] = max(dp[i], dp[j]+1);
8              ans = max(ans, 1LL*dp[i]);
9          }
10     }
11 }
12 cout<<ans;

```

n^2 的算法如果对 $1e5$ 及以上的数据来说有点慢，我们可以进行优化，可优化为 $O(n \log n)$,如下

```

1  vector<int> low(n + 1, 0);
2  int len = 0;
3  for (int i = 1; i <= n; i++)
4  {
5      if (b[i] > low[len])
6      {
7          len++;
8          low[len] = b[i];
9      }
10     else
11     {
12         int idx = lower_bound(low.begin()+1, low.begin()+len+1, b[i]) -
low.begin();
13         low[idx] = b[i];
14     }
15 }
16
17 cout << len;

```

解释: low_i 表示长度为 i 的最长上升子序列；我们从1开始遍历到 n ，如果遇到当前数组的数值大于 low 中最后的元素，我们就可以把当前的数值接到后面；如果遇到严格小于 low 中最后的元素，我们就可以将其替换到第一个大于他的位置上，可以证明，这是更优的，因为如果后面的值越小，就更容易接上更多的值

求 **LCS** 的问题部分也可转化为求 **LIS** 的问题，例如，如果对应两个数组中的元素范围相同且每个数只出现一次，我们就可以把其中一个数组当作基准来调整另一个数组中的元素；

更具体的说，如果现在给你两个数组 a, b ，他们都是 n 的排列，让你求出 a 和 b 的最长公共子序列，我们就可以以 a 为基准，对 b 进行映射，那么问题就**等价**于对映射后的 b 求最长上升子序列

如下

```
1  int n;
2      cin >> n;
3      vector<int> pos(n+1,0);
4      vector<int> b(n + 1, 0);
5      for (int i = 1; i <= n; i++)
6      {
7          int a;
8          cin>>a;
9          pos[a] = i;
10     }
11     for (int i = 1; i <= n; i++)
12     {
13         cin >> b[i];
14         b[i] = pos[b[i]];
15     }
16
17     vector<int> low(n + 1, 0);
18     int len = 0;
19     for (int i = 1; i <= n; i++)
20     {
21         if (b[i] > low[len])
22         {
23             len++;
24             low[len] = b[i];
25         }
26         else
27         {
28             int idx = lower_bound(low.begin()+1, low.begin()+len+1, b[i]) -
low.begin();
29             low[idx] = b[i];
30         }
31     }
32
33     cout << len;
34
```

3.6 搜索 && 数据结构

1 //lambda表达式:

```

2  auto dfs = [&](auto &&self,int u,int fa) -> void{
3      ....
4      //掉用时
5      self(self,v,u);
6      return;
7  };
8  //第一次调用
9  dfs(dfs,1)
10
11
12 void dfs(int u, int fa) {
13     for (int v : adj[u]) {
14         if (v == fa)
15             continue;
16         dfs(v, u);
17     }
18 }
19
20 //二者等价
21
22 void dfs(int u,int fa){
23     for(int v = 0;v<adj[u].size();u++){
24         if(adj[u][v] == fa)
25             continue;
26         dfs(adj[u][v],u);
27     }
28 }
29
30
31 //统计子树+公共祖先倍增
32
33 vector<ll> in(N),out(N); //统计进出x
34 ll tim =0;
35 vector<ll> sz(N,0); //统计子树大小
36 vector<vector<ll>> up(N,vector<ll>(LOG,0)); //倍增祖先 up[u][i]表示u向上跳 $2^i$ 
步到达的节点
37 vector<ll> depth(N,0); //统计深度
38 void dfs(int u,int fa,int d){
39     in[u] = ++tim;
40     depth[u] = d;
41     sz[u] = 1;
42
43     up[u][0] = fa;
44     for(int i = 1;i<LOG;i++){
45         if(up[u][i-1] != 0){
46             up[u][i] = up[up[u][i-1]][i-1];
47         }else{
48             up[u][i] = 0;
49         }
50     }
51
52     for(int v:adj[u]){
53         if(v == fa) continue;
54         dfs(v,u,d+1);
55         sz[u] += sz[v]; //累加子树大小
56     }
57     out[u] = ++tim;
58 }

```

```

59
60 bool isAncestor(int u,int v){ //判断v是否是u的祖先
61     return in[u] <= in[v] && out[v] <= out[u];
62 }
63
64 // 找到 v 的祖先中，是 u 的直接子节点的那个点
65 // 前提: u 是 v 的严格祖先
66 int getChildTowards(int u, int v) { //获取u-1
67
68     for (int i = LOG - 1; i >= 0; i--) {
69         // 如果跳一步之后，深度仍然比 u 大（说明还在 u 下面），就往上跳
70         if (up[v][i] != 0 && depth[up[v][i]] > depth[u]) {
71             v = up[v][i];
72         }
73     }
74     return v;
75 }

```

```

1 void bfs(int st) {
2     vector<int> vis(N, 0);
3     queue<int> q;
4     q.push(st);
5     vis[st] = 1; //标记已经访问过的
6     while (!q.empty()) {
7         int u = q.front();
8         q.pop(); //访问后移除
9         for (int v : adj[u]) {
10             if (vis[v])
11                 continue;
12             q.push(v); //将以u为节点的子节点存入队列，继续遍历
13             vis[v] = 1; //标记
14         }
15     }
16 }

```

最短路

dijkstra(处理非负权边的最短路)

标准模板

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // 使用 long long 防止路径总和爆 int
5 using ll = long long;
6
7 // 定义无穷大，注意不要用 INT_MAX，防止相加溢出
8 const ll INF = 0x3f3f3f3f3f3f3f3f;
9 // 或者直接 const ll INF = 1e18;
10
11 const int N = 100005; // 根据题目最大节点数修改

```

```

12  const int M = 200005; // 根据题目最大边数修改
13
14  // 邻接表存图: vector<pair<目标点, 权值>>
15  struct Edge {
16      int to;
17      ll w;
18  };
19  vector<Edge> adj[N];
20
21  // dist[i] 存储起点到 i 的最短距离
22  ll dist[N];
23
24  // 标记数组 (可选, 但在堆优化中用于剪枝)
25  // bool vis[N];
26
27  // n: 节点数, s: 起点
28  void dijkstra(int n, int s) {
29      // 1. 初始化距离为无穷大
30      for (int i = 1; i <= n; i++) {
31          dist[i] = INF;
32          // vis[i] = false;
33      }
34
35      // 2. 起点距离设为 0
36      dist[s] = 0;
37
38      // 3. 优先队列 (小根堆): 存储 {当前距离, 节点编号}
39      // greater 让 pair 按照 first 从小到大排序
40      priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
41      int>>> pq;
42
43      // 把起点放入队列
44      pq.push({0, s});
45
46      while (!pq.empty()) {
47          // 取出当前距离最小的点
48          auto [d, u] = pq.top();
49          pq.pop();
50
51          // 【关键剪枝】: 懒惰删除
52          // 如果当前取出的距离 d 大于已经更新过的最短距离 dist[u],
53          // 说明这个节点是旧的、无效的信息, 直接跳过。
54          if (d > dist[u]) continue;
55
56          // 如果需要 vis 数组:
57          // if (vis[u]) continue;
58          // vis[u] = true;
59
60          // 遍历 u 的所有邻居
61          for (auto& edge : adj[u]) {
62              int v = edge.to;
63              ll w = edge.w;
64
65              // 【松弛操作】: 如果经由 u 到 v 更近
66              if (dist[u] + w < dist[v]) {
67                  dist[v] = dist[u] + w;
68                  pq.push({dist[v], v}); // 将更新后的 v 放入队列
69              }
70          }
71      }
72  }

```

```

69     }
70 }
71 }
72
73 int main() {
74     // 加速 I/O
75     ios::sync_with_stdio(false);
76     cin.tie(0);
77
78     int n, m, s;
79     // 输入: 节点数, 边数, 起点
80     cin >> n >> m >> s;
81
82     // 建图
83     for (int i = 0; i < m; i++) {
84         int u, v;
85         ll w;
86         cin >> u >> v >> w;
87         // 有向图
88         adj[u].push_back({v, w});
89
90         // 如果是无向图, 加上下面这句:
91         // adj[v].push_back({u, w});
92     }
93
94     // 运行算法
95     dijkstra(n, s);
96
97     // 输出结果
98     for (int i = 1; i <= n; i++) {
99         if (dist[i] == INF) {
100             cout << -1 << " "; // 无法到达
101         } else {
102             cout << dist[i] << " ";
103         }
104     }
105     cout << endl;
106
107     return 0;
108 }

```

全源最短路(处理含负权边) (SPFA)

- 核心思想: 首先创建一个虚拟超级 0 节点, 向所有节点连一条权值为 0 的边, 跑一次 *SPFA* 求出势能函数后, 利用他把所有边转换成非负权边, 接着按照题意把要求的最短路用 *dijkstra* 求出即可

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  #define endl '\n'
5
6  const ll INF = 1e18; // 设置一个足够大的无穷大值, 防止累加时溢出
7
8  // 边的结构体

```

```

9  struct node {
10     ll to; // 目标节点
11     ll w;  // 边权
12 };
13
14  int main() {
15     // -----
16     // 0. 基础设置与建图
17     // -----
18     ios::sync_with_stdio(0); // 优化 C++ 输入输出流, 防止大数据读写超时
19     cin.tie(0);
20
21     ll n, m;
22     cin >> n >> m;
23
24     // 使用 vector 的 vector, 兼容所有 C++ 标准, 避免 VLA (变长数组) 报错
25     vector<vector<node>> adj(n + 1);
26
27     for (int i = 1; i <= m; i++) {
28         ll u, v, w;
29         cin >> u >> v >> w;
30         adj[u].push_back({v, w}); // 构建有向图
31     }
32
33     // -----
34     // 1. SPFA 求势能数组 h (隐式建立虚拟节点 0)
35     // -----
36     vector<ll> cnt(n + 1, 0); // cnt[i] 记录节点 i 入队的次数, 用于判负环
37     vector<bool> vis(n + 1, false); // vis[i] 表示节点 i 当前是否在队列中 (SPFA
核心)
38
39     // h 初始必须为 0! 等价于虚拟节点 0 到各点的距离为 0
40     vector<ll> h(n + 1, 0);
41     queue<ll> q;
42
43     // 隐式从虚拟源点出发: 将所有真实节点压入队列, 初始距离(势能)全为 0
44     for (int i = 1; i <= n; i++) {
45         q.push(i);
46         vis[i] = true; // 标记已在队列中
47         cnt[i] = 1;    // 相当于所有点已经入队 1 次
48     }
49
50     // SPFA 主循环
51     while (!q.empty()) {
52         auto u = q.front();
53         q.pop();
54         vis[u] = false; // 节点出队, 取消标记
55
56         for (auto &edge : adj[u]) {
57             ll v = edge.to;
58             ll w = edge.w;
59
60             // 松弛操作
61             if (h[v] > h[u] + w) {
62                 h[v] = h[u] + w;
63
64                 // 只有当 v 不在队列中时, 才需要将其入队
65                 if (!vis[v]) {

```

```

66         vis[v] = true;
67         cnt[v]++; // 记录入队次数
68
69         // 如果某个点入队次数超过图的总节点数 n，说明在无限绕负权环
70         if (cnt[v] > n) {
71             cout << -1 << endl;
72             return 0; // 发现负环，直接结束程序
73         }
74
75         //q.push 必须放在 if(!vis[v]) 内部！防止重复无意义入队
76         q.push(v);
77     }
78 }
79 }
80 }
81
82 //将所有边转换为非负
83 for(int i = 1; i <= n; i++){
84     for(auto &edge : adj[i]){
85         edge.w = edge.w + h[i] - h[edge.to];
86     }
87 }
88
89 //后面根据题目进行dijkstra即可，注意最后要对边权进行还原
90 //真实 w = dist[j] + h[j] - h[i]
91
92 return 0;
93 }

```

分册图+回溯寻找路径

[P1266 [BalticOI 2002](#)] [速度限制 - 洛谷](#)

1. 分册图

- **分层图最短路**，本质上是**图论与动态规划（DP）的结合**（所以也常被称为“状态机最短路”）。
- 因为到达某一个节点时会因为上一个节点的结果不同而可能导致当前节点的结果不同，那么这种情况下我们需要对 *dist* 数组进行升维，二维或者三维，根据题目限制条件而定
 - 一般题目带有“**状态/油量/速度/免费次数/打折券**”等带有限制性条件时，一般都需要分层图

2. 回溯找最短路对应的节点

- 可根据 *dist* 创建对应数量和维度的 *pre* 数组，用来存放谁更新了当前最优的状态，最后倒序查找即可

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  #define endl '\n'
5
6  const ll INF = 1e18;
7
8  struct Node{
9      int to;

```



```

10     int l;
11     int v;
12 };
13
14 int main(){
15     ios::sync_with_stdio(0);
16     cin.tie(0);
17     int n,m,d;
18     cin>>n>>m>>d;
19     vector<Node> adj[n];
20     for(int i = 1;i<=m;i++){
21         int u,v,s,l;
22         cin>>u>>v>>s>>l;
23         adj[u].push_back({v,l,s});
24     }
25
26     //本题的边权，也就是时间并非固定不变，所有需要再开一维来进行表示，dist[u][se]表示到
    达u节点且速度为se时的最短路径
27     vector<vector<double>> dist(n,vector<double>(505,INF));
28
29     priority_queue<tuple<double,ll,ll>,vector<tuple<double,ll,ll>>,greater<tuple<double,ll,ll>>> pq;
30     vector<vector<int>> pre_pos(n,vector<int>(505,-1)); //表示到达对应状态的上一
    个路口
31     vector<vector<int>> pre_sp(n,vector<int>(505,-1)); //表示到达对应状态的上一个
    速度
32     pq.push({0.0,70,0});
33     dist[0][70] = 0;
34
35     while(!pq.empty()){
36         auto [t,se,u] = pq.top();
37         pq.pop();
38
39         if(t>dist[u][se]) continue;
40
41         for(auto &edge : adj[u]){
42             int v = edge.to;
43             int l = edge.l;
44             int se1 = edge.v;
45             double t1 = 0.0;
46
47             if(se1 == 0){
48                 t1 = (double)l / se;
49                 se1 = se;
50             }else{
51                 t1 = (double)l / se1;
52             }
53
54             if(dist[v][se1]>dist[u][se]+t1){
55                 dist[v][se1] = dist[u][se] + t1;
56                 pq.push({dist[v][se1],se1,v});
57                 pre_pos[v][se1] = u;
58                 pre_sp[v][se1] = se;
59             }
60         }
61     }
62
63     double minn = INF;

```

```

63     vector<int> ans;
64     int sp = -1;
65     for(int i = 0; i <= 500; i++){
66         if(minn > dist[d][i]){
67             minn = dist[d][i];
68             sp = i;
69         }
70     }
71
72     int cur_u = d;
73     int cur_sp = sp;
74
75     while(cur_u != -1){
76         ans.push_back(cur_u);
77
78         int next_u = pre_pos[cur_u][cur_sp];
79         int next_sp = pre_sp[cur_u][cur_sp];
80
81         cur_u = next_u;
82         cur_sp = next_sp;
83     }
84     reverse(ans.begin(), ans.end());
85
86     for(int i = 0; i < ans.size(); i++){
87         cout << ans[i] << " ";
88     }
89
90     return 0;
91 }

```

Floyd

该算法可求任意两点的之间的最短长度，但复杂度过高，一般只在n较小的情况下使用或优化

```

1  for (k = 1; k <= n; k++) {
2      for (x = 1; x <= n; x++) {
3          for (y = 1; y <= n; y++) {
4              f[x][y] = min(f[x][y], f[x][k] + f[k][y]);
5          }
6      }
7  } //f[x][y]表示x到y的最短路程

```

并查集

专门用来处理一些不相交集合并与查询问题

```

1  /**
2   * 并查集（DSU）模板
3   * 包含：路径压缩 + 按大小合并
4   * 复杂度：O(alpha(n)) ≈ O(1)
5   */
6  struct DSU {

```

```

7      std::vector<int> parent;
8      std::vector<int> siz; // 记录每个集合的大小
9      int count;           // 记录连通分量的数量
10
11     // 初始化: n 为节点数量
12     DSU(int n) : parent(n + 1), siz(n + 1, 1), count(n) {
13         // 初始时每个节点的父节点是自己
14         std::iota(parent.begin(), parent.end(), 0);
15     }
16
17     // 查找 (Find) - 路径压缩
18     int find(int x) {
19         // 如果 x 不是根节点, 递归找根, 并进行路径压缩
20         return parent[x] == x ? x : parent[x] = find(parent[x]);
21     }
22
23     // 合并 (Union) - 按大小合并
24     // 返回值: true 表示合并成功 (原本不在一组), false 表示原本就在一组
25     bool merge(int x, int y) {
26         int rootX = find(x);
27         int rootY = find(y);
28
29         if (rootX == rootY) return false; // 已经在同一个集合
30
31         // 启发式合并: 把小的集合合并到大的集合上, 保持树的高度较低
32         if (siz[rootX] < siz[rootY]) std::swap(rootX, rootY);
33
34         parent[rootY] = rootX; // Y 挂到 X 上
35         siz[rootX] += siz[rootY]; // 更新 X 的大小
36         count--; // 连通分量减少一个
37         return true;
38     }
39
40     // 判断是否连通
41     bool connected(int x, int y) {
42         return find(x) == find(y);
43     }
44
45     // 获取某个节点所在集合的大小
46     int getSize(int x) {
47         return siz[find(x)];
48     }
49 };

```

树状数组

树状数组是一种支持 **单点修改** 和 **区间查询** 的, 代码量小的数据结构。

树状数组利用数的**二进制特征**来定义“管辖范围”, 即一个数的二进制的最后一位 1 以及其后所有的 0 所构成的二进制大小, 就是该数所管辖的区间范围。例如 8 的二进制为 1000, 那么 a[8] 所管辖的范围就是 1-8 这个区间, 再例如 7, a[7] 所管辖的范围就只有 7

lowbit(x) 用于提取 x 在二进制表示下最低位的 1 及其后面的 0 构成的数值。

```

1 struct BIT {

```

```

2   int n;
3   vector<ll> tree;
4   BIT(int n) : n(n), tree(n + 1, 0) {}
5
6   ll lowbit(int x){
7       return x & (-x);
8   }
9
10  // 在位置 i 增加 delta
11  void add(int i, ll delta) {
12      for (; i <= n; i += lowbit(i)) {
13          tree[i] = tree[i] + delta;
14      }
15  }
16
17  // 查询 1 到 i 的区间和
18  ll query(int i) {
19      ll sum = 0;
20      for (; i > 0; i -= lowbit(i)) {
21          sum = sum + tree[i];
22      }
23      return sum;
24  }
25
26  // 查询 l 到 r 的区间和
27  ll query_range(int l, int r) {
28      if (l > r) return 0;
29      return query(r) - query(l - 1);
30  }
31 };
32

```

线段树

区间修改+区间查询

```

1   ll arr[MAXN];      // 原数组
2   ll tree[MAXN * 4]; // 线段树数组
3   ll tag[MAXN * 4];  // 懒标记(加)
4   vector<ll> tag1(MAXN*4,1); // 懒标记(乘)
5
6   // 向上更新 (Push Up): 用子节点算父节点
7   void push_up(ll p) {
8       tree[p] = tree[p << 1] + tree[p << 1 | 1];
9   }
10
11  // 向下下放 (Push Down), 每当要访问数据的时候都要下放
12  // p: 当前节点, len: 当前节点管辖的区间长度
13  void push_down(ll p, ll len) {
14      //如果存在修改乘上某值, 需要先对乘法的标记进行下放
15      if(tag1[p] != 1){
16          tag1[p<<1] *= tag1[p];
17          tag1[p<<1 | 1] *= tag1[p];
18
19          //需要对加法的标记进行倍增

```

```

20     tag[p<<1] *= tag1[p];
21     tag[p<<1 | 1] *= tag1[p];
22
23     tree[p<<1] *= tag1[p];
24     tree[p<<1 | 1] *= tag1[p];
25
26     tag1[p] = 1;
27 }
28 if (tag[p]) {
29     // 1. 传给左子 (p<<1) 和 右子 (p<<1|1)
30     tag[p << 1] += tag[p];
31     tag[p << 1 | 1] += tag[p];
32
33     // 2. 更新子节点的值 (增量 = tag * 区间长度)
34     tree[p << 1] += tag[p] * (len - len / 2);
35     tree[p << 1 | 1] += tag[p] * (len / 2);
36
37     // 3. 清除当前标记
38     tag[p] = 0;
39 }
40 }
41
42 // 建树
43 void build(ll p, ll l, ll r) {
44     tag[p] = 0;
45     if (l == r) {
46         tree[p] = arr[l];
47         return;
48     }
49     ll mid = (l + r) >> 1;
50     build(p << 1, l, mid);
51     build(p << 1 | 1, mid + 1, r);
52     push_up(p);
53 }
54
55 // 区间修改: [ql, qr] 范围全加 k
56 void update(ll p, ll l, ll r, ll ql, ll qr, ll k) {
57     // 1. 完全覆盖, 直接更新并打标, 不递归
58     if (ql <= l && r <= qr) {
59         tree[p] += k * (r - l + 1);
60         tag[p] += k;
61         return;
62     }
63
64     // 2. 未完全覆盖, 先下放标记, 再递归
65     push_down(p, r - l + 1);
66     ll mid = (l + r) >> 1;
67     if (ql <= mid) update(p << 1, l, mid, ql, qr, k);
68     if (qr > mid) update(p << 1 | 1, mid + 1, r, ql, qr, k);
69     push_up(p);
70 }
71
72 void update1(ll p, ll l, ll r, ll ql, ll qr, ll k) {
73     if (ql <= l && r <= qr) {
74         tree[p] *= k;
75         tag1[p] *= k;
76         tag[p] *= k;
77         return;

```

```

78     }
79
80     push_down(p,r-1+1);
81     ll mid = (l+r)>>1;
82     if(q1 <= mid) update(p<<1,l,mid,q1,q1,k);
83     if(qr>mid) update(p<<1 | 1,mid+1,r,q1,q1,k);
84     push_up(p);
85 }
86
87 // 区间查询：求 [q1, qr] 的和
88 ll query(ll p, ll l, ll r, ll q1, ll qr) {
89     if (q1 <= l && r <= qr) return tree[p];
90
91     push_down(p, r - l + 1); // 查之前也要下放
92     ll mid = (l + r) >> 1;
93     ll res = 0;
94     if (q1 <= mid) res += query(p << 1, l, mid, q1, qr);
95     if (qr > mid) res += query(p << 1 | 1, mid + 1, r, q1, qr);
96     return res;
97 }

```

3.7 数学

一些定理？以及一些遇到的数学技巧

唯一分解定理

是指大于1的正整数 n ，都可以唯一地表示为有限个素数的乘积

$$n = p_1^{a_1} \times p_2^{a_2} \times \dots \times p_n^{a_n}$$

叉积求三角形面积

1. 利用向量的叉积

考虑空间坐标系 $Oxyz$, $\overrightarrow{OA} = (x_1, y_1, 0)$, $\overrightarrow{OB} = (x_2, y_2, 0)$, 则

$$\begin{aligned}
 S_{\triangle OAB} &= \frac{1}{2} |OA| \cdot |OB| \cdot \sin \angle AOB = \frac{1}{2} \left| \overrightarrow{OA} \times \overrightarrow{OB} \right| \\
 &= \frac{1}{2} \left| \begin{bmatrix} i & j & k \\ x_1 & y_1 & 0 \\ x_2 & y_2 & 0 \end{bmatrix} \right| = \frac{1}{2} \left| \begin{vmatrix} x_1 & y_1 \\ x_2 & y_2 \end{vmatrix} k \right| \\
 &= \frac{1}{2} |x_1 y_2 - x_2 y_1|.
 \end{aligned}$$

2. 推广

对于一般的 $\triangle ABC$, 设点 $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$, 则

$$\begin{aligned}\overrightarrow{AB} &= (x_2 - x_1, y_2 - y_1), \\ \overrightarrow{AC} &= (x_3 - x_1, y_3 - y_1),\end{aligned}$$

所以

$$S_{\triangle ABC} = \frac{1}{2} \left| \det \begin{bmatrix} x_2 - x_1 & y_2 - y_1 \\ x_3 - x_1 & y_3 - y_1 \end{bmatrix} \right|.$$

3.7.1 快速幂

快速幂是求解的问题，其中a,b限定为整。如求3的 $1e18$ 次方，直接递推肯定超时。

原理：如求3的8次方，我们可以先算3的2次，再2次乘2次，到3的4次，4次乘4次，到3的8次

```
1  #include <iostream>
2  #include <bitset>
3  #include <cmath>
4  using namespace std;
5  using ll = long long;
6  ll qpow(ll a, ll b)
7  {
8      ll res = 1;
9      while (b != 0)
10     {
11         if (b % 2 == 1)
12             res *= a;
13         a *= a;
14         b /= 2;
15     }
16     return res;
17 }
18 int main()
19 {
20     ll a, b;
21     cin >> a >> b;
22     cout << qpow(a, b) << endl;
23     return 0;
24 }
```

3.7.2 矩阵快速幂

问题：快速求解 $n*n$ 的矩阵A，求 A^b

```
1  #include <iostream>
2  #include <bitset>
3  #include <cmath>
4  #include <vector>
5  using namespace std;
6  using ll = long long;
7  const ll mod = 1e9 + 7;
```

```

8  vector<vector<ll>> mul(const vector<vector<ll>> &a, const vector<vector<ll>>
    &b) // 矩阵相乘
9  {
10     ll n = a.size() - 1;
11     vector<vector<ll>> res(n + 1, vector<ll>(n + 1, 0));
12     for (int i = 1; i <= n; ++i)
13     {
14         for (int j = 1; j <= n; ++j)
15         {
16             for (int k = 1; k <= n; ++k)
17             {
18                 res[i][j] = (res[i][j] + a[i][k] * b[k][j] % mod + mod) %
mod;
19             }
20         }
21     }
22     return res;
23 }
24 vector<vector<ll>> qpow(vector<vector<ll>> a, ll b) // 矩阵快速幂
25 {
26     ll n = a.size() - 1;
27     vector<vector<ll>> res(n + 1, vector<ll>(n + 1, 0));
28     for (int i = 1; i <= n; ++i)
29         res[i][i] = 1;
30     while (b)
31     {
32         if (b % 2)
33             res = mul(res, a);
34         a = mul(a, a);
35         b >>= 1;
36     }
37     return res;
38 }
39 int main()
40 {
41     ll n, b;
42     cin >> n >> b; // 矩阵阶数n,次数b.
43     vector<vector<ll>> a(n + 1, vector<ll>(n + 1, 0));
44     for (int i = 1; i <= n; ++i)
45     {
46         for (int j = 1; j <= n; ++j)
47         {
48             cin >> a[i][j];
49         }
50     }
51     vector<vector<ll>> res = qpow(a, b);
52     for (int i = 1; i <= n; i++)
53     {
54         for (int j = 1; j <= n; ++j)
55         {
56             cout << res[i][j] << " ";
57         }
58         cout << endl;
59     }
60     return 0;
61 }

```


3.7.3 高精度

1. 加法高精度

```
1 //计算 123456789 + 987654321
2 /*
3 高精度的加法思想
4 1.把大数存到字符串;
5     2.字符串的每个字符数字都通过ASCII转换存到数组,
6     注意的是要低位存在数组开头:a[i] = s[len-i-1]-'0';
7
8     3.获取最大的数长度:max(len1,len2) ;
9     4.把a,b值加入到c数组: c[i] = a[i]+b[i];
10
11     5.c数组加法进位的算式:
12     ① c[i+1] += c[i]/10;
13     ② c[i] %= 10;
14
15     6.数字溢出,长度+1;
16     7.反向输出结果;
17 */
18 #include<iostream>
19 #include<string>
20 using namespace std;
21 string s1,s2;
22 int a[10000],b[10000],c[10000];
23 int main(){
24     // 1.输入值,长度
25     cin>>s1>>s2;
26     int len1 = s1.size();
27     int len2 = s2.size();
28     // 2.把字符转为整数存到数组
29     // 注意要个位存到数组开头
30     for(int i=0;i<len1;i++){
31         a[i] = s1[len1-i-1]-'0';
32     }
33     for(int i=0;i<len2;i++){
34         b[i] = s2[len2-i-1]-'0';
35     }
36     // 3.获取最大的数。
37     int len = max(len1,len2);
38     // 对各个位数进行相加
39     for(int i=0;i<len;i++){
40         c[i]=a[i]+b[i];
41     }
42     //4.进位
43     for(int i=0;i<len;i++){
44         c[i+1] += c[i]/10;
45         c[i] %= 10;
46     }
47     //5.溢出
48     while(c[len]==0 && len>0){
49         len--;
50     }
51     if(c[len]>0){
```

```

52         len++;
53     }
54     //6. 反向输出
55     for(int i=len-1;i>=0;i--){
56         cout<<c[i];
57     }
58     return 0;
59 }

```

2. 高精度减法

```

1  // 辅助函数：判断 s1 是否小于 s2
2  bool isSmaller(const string& s1, const string& s2) {
3      if (s1.size() != s2.size()) {
4          return s1.size() < s2.size();
5      }
6      return s1 < s2; // 长度相同时直接用字典序比较
7  }
8
9  ll solve(string s1, string s2) {
10     // 1. 修复比较逻辑，不用 stoi
11     if (isSmaller(s1, s2)) {
12         swap(s1, s2);
13     }
14
15     // 2. 修复数组初始化，使用 vector 自动初始化为 0，防止垃圾值
16     // 开大一点防止溢出
17     int len1 = s1.size();
18     int len2 = s2.size();
19     vector<int> a(len1, 0);
20     vector<int> b(len1, 0); // 让 b 的大小和 a 一样，方便减法，不足补0
21     vector<int> c(len1, 0);
22
23     for (int i = 0; i < len1; i++) a[i] = s1[len1 - i - 1] - '0';
24     for (int i = 0; i < len2; i++) b[i] = s2[len2 - i - 1] - '0';
25
26     // 减法逻辑
27     for (int i = 0; i < len1; i++) {
28         if (a[i] < b[i]) {
29             a[i + 1]--;
30             a[i] += 10;
31         }
32         c[i] = a[i] - b[i];
33     }
34
35     // 去除前导零
36     int real_len = len1;
37     while (real_len > 1 && c[real_len - 1] == 0) {
38         real_len--;
39     }
40
41     // 3. 修复返回值：直接在计算过程中取模，而不是转成 stoi
42     // 结果现在存在 c[0]...c[real_len-1] 中，c[0] 是个位
43     ll num = 0;
44     // 从高位到低位还原数值并取模（秦九韶算法）

```

```

45     for (int i = real_len - 1; i >= 0; i--) {
46         num = (num * 10 + c[i]) % MOD;
47     }
48
49     return num;
50 }

```

3. 高精度乘法

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4  const int MAXN = 40500; // 最大长度
5  int a[MAXN], b[MAXN], c[MAXN];
6  int main() {
7      string s1, s2;
8      cin >> s1 >> s2;
9      int n = s1.length(), m = s2.length(), len = n + m;
10     // 逆序存储
11     for (int i = 0; i < n; i++) a[n - i] = s1[i] - '0';
12     for (int i = 0; i < m; i++) b[m - i] = s2[i] - '0';
13     // 累加乘积
14     for (int i = 1; i <= n; i++) {
15         for (int j = 1; j <= m; j++) {
16             c[i + j - 1] += a[i] * b[j];
17         }
18     }
19     // 处理进位
20     for (int i = 1; i < len; i++) {
21         if (c[i] >= 10) {
22             c[i + 1] += c[i] / 10;
23             c[i] %= 10;
24         }
25     }
26     // 删除前导零并输出
27     while (len > 1 && c[len] == 0) len--;
28     for (int i = len; i > 0; i--) cout << c[i];
29     return 0;
30 }

```

3.7.4离散化

1. **离散化差分** (Discretized Difference Array) 是解决“**大范围坐标、少操作次数**”区间问题的核心算法。

简单来说，就是当题目中告诉你坐标范围是 $1 \leq x \leq 10^9$ ，但操作次数只有 **N** 只有 10^5 时，因为内存和时间限制，你不能开一个 10^9 大小的数组。此时，我们需要把**用不到的中间空白坐标压缩掉**，或者**只存储有变化的坐标点**。

核心思想：

数轴是连续的，但数值的变化是**离散**的。

只有在区间的**起点和终点**，覆盖层数才会发生突变。在两个相邻的“关键点”之间，覆盖层数是恒定的。

因此，我们只需要记录这些**关键点（变化点）**，然后计算两个关键点之间的距离乘以当前的层数，就是这段区间的贡献。

```
1 map<ll, int> diff;
2 // 1. 读入并差分
3 for(int i=0; i<n; i++) {
4     cin >> l >> r;
5     diff[l]++;
6     diff[r+1]--; // 注意这里通常是 r+1, 代表左闭右开区间 [l, r]
7 }
8
9 // 2. 扫描线统计
10 ll ans = 0, sum = 0, pre = -1;
11 for(auto& [pos, val] : diff) {
12     // 第一次循环只记录起点, 不计算
13     if(pre != -1) {
14         // 计算上一段 [pre, pos) 的长度
15         ll len = pos - pre;
16         // 根据 sum (层数) 判断是否计入答案
17         if(sum > 0) ans += len;
18     }
19     sum += val; // 更新层数
20     pre = pos; // 更新上一个点
21 }
```

3.7.5 欧拉筛

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4 using ll = long long;
5 const ll maxn = 1e6;
6 vector<ll> prime; // 存储已经找到的所有素数 (相当于“素数字典”);
7 vector<ll> phi(maxn + 1, 1); // 存储每个数的欧拉函数值 (顺带计算)
8 vector<bool> vis(maxn + 1, 0); // 标记某个数是否为合数 (false= 素数, true = 合数)
9 void init()
10 {
11     phi[1] = 1;
12     vis[1] = 1; // 1既不是质数也不是合数, 但在筛法中标记为1避免重复处理
13     for (ll i = 2; i <= maxn; ++i)
14     {
15         if (!vis[i])
16         {
17             prime.push_back(i);
18             phi[i] = i - 1;
19         }
20         for (int j = 0; j < prime.size(); ++j)
21         {
22             ll p = prime[j];
```

```

23         if (p * i > maxn)
24             break;
25         vis[p * i] = true;
26         if (i % p == 0)
27         {
28             phi[i * p] = phi[i] * p;
29             break;
30         }
31         else
32         {
33             phi[i * p] = phi[i] * phi[p];
34         }
35     }
36 }
37 }
38 int main()
39 {
40     init();
41     for (int i = 1; i <= 10; i++)
42     {
43         cout << phi[i] << endl;
44     }
45     return 0;
46 }

```

3.7.6 逆元 取模 最大公约数(gcd) 最小公倍数(lcm)

取模最主要的是要注意步步取模，避免爆值，例如

- **加法:** $(a+b) \% \text{MOD}$
- **乘法:** $(a * b) \% \text{MOD}$
- **减法:** $(a - b + \text{MOD}) \% \text{MOD}$ (注意减法要先加上模数后再取模，避免出现负值取模)
- **除法:** 除法不能直接取模，需要求模逆元；除以一个数 b 等于乘以 b 的模逆元

$\frac{a}{b} (\text{mod } P) \equiv a * b^{P-2} (\text{mod } P)$ 可以用快速幂计算 `qpow(b, MOD-2)` (模数得是质数才成立，但一般题目给出的都是质数)

1到1e6模1e9+7的逆元模板

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  using ll = long long;
5  const ll mod = 1e9 + 7;
6  const ll maxn = 1e6;
7  vector<ll> inv(maxn + 1, 1);
8  void init()
9  {
10     for (int i = 2; i <= maxn; ++i)
11     {
12         inv[i] = (mod - mod / i * inv[mod % i] % mod) % mod;
13     }
14 }

```

```

15 int main()
16 {
17     init();
18     for (int i = 1; i <= 10; ++i)
19     {
20         cout << inv[i] << endl;
21     }
22     return 0;
23 }

```

```

1  ll gcd(ll a, ll b)
2  {
3      a = abs(a);
4      b = abs(b);
5      if (a > b)
6          swap(a, b);
7      if (a == 0)
8          return b;
9      else
10         return gcd(b % a, a);
11 }
12 ll lcm(ll a, ll b)
13 {
14     return a / gcd(a, b) * b; // 先除后乘，尽量不溢
15 }

```

3.7.7排列数和组合数

```

1  vector<ll> fact(n+1), inv(n+1); //阶乘和逆元
2
3  ll qpow(ll a, ll b)
4  {
5      ll res = 1;
6      while (b != 0)
7      {
8          if (b % 2 == 1)
9              res = res * a % MOD;
10         a = a * a % MOD;
11         b /= 2;
12     }
13     return res;
14 }
15
16 void init(){
17     fact[0] = 1;
18     for(int i = 1; i <= n; i++){
19         fact[i] = (fact[i-1] * i) % MOD;
20     }
21     inv[n] = qpow(fact[n], MOD-2);
22     for(int i = n - 1; i >= 0; i--){
23         inv[i] = inv[i+1] * (i+1) % MOD;
24     }

```

```

25 }
26
27 ll C(ll n, ll k){//n中选k
28     if(k<0 || k>n) return 0;
29     return fact[n] * inv[k] % MOD * inv[n-k] % MOD;
30 }
31
32 ll A(ll n, ll k){
33     if(k<0 || k>n) return 0;
34     return fact[n] * inv[n-k] % MOD;
35 }

```

位运算

```

1  __builtin_ctz(x) //获取x二进制后缀0的长度
2  __builtin_clz(x) //获取x二进制前缀0的长度 //二者皆为接受int类型，接受ll需要后面加ll
   //

```

3.8 dp

4. 一些类型题的处理思路

4.1 存在大量插入，删除以及查询全局最小(大)值的

1. 方法一：使用multiset
 - 适用场景：总操作次数(插入+删除)在 10^5 级别，且时间为 2 秒可用
 - 插入：`ms.insert(x)`，删除一个元素：`ms.erase(ms.find(x))` 均为 $O(\log n)$
2. 方法二：双优先队列
 - 适用场景：极高频的插入和删除，且只关心最大值或最小值
 - 使用：定义两个最小堆的pq，一个存放所有插入的元素，一个存放所有删除的元素，查询时，如果两个堆顶的元素都相同，说明该值已被删除，对两个堆执行 *pop* 操作,直到不相同为止，该次查询的值就是负责存放的堆的堆顶元素
3. 方法三：动态开点线段树

5. 优化技巧

线段树优化建图

如果遇到 **点到区间** 连边或 **区间到点** 连边，且总边数和总操作数都很大的时候，且需要跑**最短路**等算法的时候一般都会使用这种技巧

例： [Problem - B - Codeforces](#)

```

1  #include <bits/stdc++.h>
2  using namespace std;

```

```

3 using ll = long long;
4 #define endl '\n'
5 #define pll pair<ll,ll>
6 #define T tuple<ll,ll,ll>
7
8 const ll INF = 1e18;
9
10 struct Node{
11     ll to;
12     ll w;
13 };
14
15 void solve()
16 {
17     ll n,q,s;
18     cin>>n>>q>>s;
19     vector<vector<Node>> adj(4*n);
20     vector<ll> in(4*n,0); //入树，点到区间
21     vector<ll> out(4*n,0); //出树，区间到点
22     ll cnt = n; //虚拟节点编号需大于所有真实点
23
24     //建树过程
25     auto build = [&](auto &&self,ll p,ll l,ll r) -> void{
26         if(l == r){
27             in[p] = 1;
28             out[p] = 1;
29             return;
30         }
31
32         ll ls = p<<1;
33         ll rs = p<<1 | 1;
34         ll mid = (l+r)>>1;
35
36         in[p] = ++cnt;
37         out[p] = ++cnt;
38
39         self(self,ls,l,mid);
40         self(self,rs,mid+1,r);
41
42         adj[in[p]].push_back({in[ls],0});
43         adj[in[p]].push_back({in[rs],0});
44
45         adj[out[ls]].push_back({out[p],0});
46         adj[out[rs]].push_back({out[p],0});
47     };
48
49     //点到区间连边
50     auto v_to_range = [&](auto &&self,ll p,ll v,ll w,ll l,ll r,ll ql,ll qr)
->void{
51         if(ql<=l && r<=qr){
52             adj[v].push_back({in[p],w});
53             return;
54         }
55
56         ll ls = p << 1;
57         ll rs = p << 1 | 1;
58         ll mid = (l + r) >> 1;
59

```



```

60         if(q1<=mid) self(self,ls,v,w,l,mid,q1,q1,qr);
61         if(qr>mid) self(self,rs,v,w,mid+1,r,q1,qr);
62     };
63
64     auto range_to_v = [&](auto &&self,ll p,ll v,ll w,ll l,ll r,ll q1,ll qr)
-> void{
65         if(q1<=l && r<=qr){
66             adj[out[p]].push_back({v,w});
67             return;
68         }
69
70         ll ls = p << 1;
71         ll rs = p << 1 | 1;
72         ll mid = (l + r) >> 1;
73
74         if(q1<=mid) self(self,ls,v,w,l,mid,q1,qr);
75         if(qr>mid) self(self,rs,v,w,mid+1,r,q1,qr);
76     };
77
78     build(build,1,1,n);
79
80     while(q--){
81         ll t;
82         cin>>t;
83         if(t==1){
84             ll v,u,w;
85             cin>>v>>u>>w;
86             adj[v].push_back({u,w});
87         }else{
88             ll v,l,r,w;
89             cin>>v>>l>>r>>w;
90             if(t==2){
91                 v_to_range(v_to_range,1,v,w,1,n,l,r);
92             }else if(t == 3){
93                 range_to_v(range_to_v,1,v,w,1,n,l,r);
94             }
95         }
96     }
97
98     vector<ll> dist(cnt+1,INF);
99     priority_queue<p11,vector<p11>,greater<p11>> pq;
100     pq.push({0,s});
101     dist[s] = 0;
102
103     while(!pq.empty()){
104         auto [d,u] = pq.top();
105         pq.pop();
106
107         if(d>dist[u]) continue;
108
109         for(auto &edge : adj[u]){
110             ll v = edge.to;
111             ll w = edge.w;
112             if(dist[v]>dist[u]+w){
113                 dist[v] = dist[u] + w;
114                 pq.push({dist[v],v});
115             }
116         }

```

```

117     }
118
119     for(int i = 1; i <= n; i++){
120         if(dist[i] == INF){
121             cout << -1 << " ";
122         } else {
123             cout << dist[i] << " ";
124         }
125     }
126
127 }
128
129 int main()
130 {
131     ios_base::sync_with_stdio(false);
132     cin.tie(NULL);
133     int t = 1;
134     // cin >> t;
135
136     while (t--)
137     {
138         solve();
139     }
140     return 0;
141 }

```

小点

1 | `ceil(x)` // 取第一个不小于 x 的整数 `<cmath>`

```

1  //加速
2  ios::sync_with_stdio(false);
3  cin.tie(0);
4  cout.tie(0);
5
6  // 2. 去重 unique 会将重复的元素移到末尾, 返回去重后最后一个有效元素的下一个位置的迭代器
7  auto last = unique(a.begin(), a.end());
8  a.erase(last, a.end()); // 配合 vector::erase s
9  a.erase(unique(a.begin(), a.end()), a.end()); /
10
11
12 // 用 endl 再极端情况下可能会 TLE, 建议直接使用 '\n'
13 // 或者直接 #define endl '\n'
14
15 // row+i 表示同一副对角线
16 // row-i 表示同一主对角线 因为会出现负值, 所以我们通常会加一个常数,
17 // row-i+n
18
19
20 // 整数向上取整可以 (a+b-1)/b
21
22 对于矩阵中的任意 2*2 子块, 想要其合为合数, 只需顺序填入值

```

$i \equiv j \pmod{x}$ 意味着 i 和 j 关于 x 同余, 那么集合可表示为 $\{1 + x, 1 + 2x, \dots, 1 + kx\}$

0-base 下 $[L, R]$ 的和用前缀和表示为 `pre[R+1] - pre[L]`

1-base 下 $[L, R]$ 的和用前缀和表示为 `pre[R] - pre[L-1]`

只由0或1组成的二维矩阵, 只要该点为1(或0)且1(或0)的总数大于等于2时, 总能找到一个终点, 使得路径上排成的字符为回文串

对于一组升序排序的数组, 如果任意两个数之差小于等于某一个值, 那么整个数组中的数据可以被两两配对, 且之差不大于那个值

好用函数

`std::is_sorted(a.begin(), a.end());` //判断数组是否是升序的 既检查是否满足 $a_i \leq a_{i+1}$, 是返回true, 否则返回false

如果想检查是否为降序可添加 `std::greater<T>()`

`std::is_sorted_until` 该函数返回一个迭代器, 返回到哪个元素开始不再是升序